Implementação do MVP Pedidos Veloz

Relatório técnico prático | Cloud DevOps | 28 de outubro de 2026

Objetivo e arquitetura

O artefato entrega uma aplicação de comércio eletrônico mínima para demonstrar o ciclo de desenvolvimento e operação. O gateway HTTP encaminha chamadas para quatro processos: pedidos, pagamentos simulados, estoque e PostgreSQL. Pedidos e estoque gravam no banco; reservas fazem atualização condicional atômica para impedir saldo negativo. O objetivo é funcionalidade educacional reproduzível, não processamento real de pagamentos ou arquitetura pronta para produção financeira.

Componente	Responsabilidade	Verificação
Gateway :5000	Entrada HTTP e encaminhamento	/health, /metrics
Pedidos :5001	Validar/criar/listar pedidos	PostgreSQL, readiness do banco
Pagamentos :5002	Validação e aprovação simulada	Resposta JSON e métricas
Estoque :5003	Consulta e reserva persistente	SQL atômico, readiness do banco
PostgreSQL :5432	Persistência de pedidos e estoque	Healthcheck pg_isready, volume/PVC

Ambiente local com Compose

`docker-compose.yml` sobe a stack com uma rede bridge privada, volume nomeado do PostgreSQL, inicialização do schema, credenciais via arquivo de ambiente e healthchecks encadeados. Somente o gateway publica porta no host; dependências esperam a prontidão real dos serviços. Isso reduz divergência local e substitui instruções manuais por um fluxo simples:

```
Copy-Item .env.example .env
docker compose config --quiet
docker compose up --build -d
docker compose ps
```

O README registra chamadas PowerShell, logs, testes e encerramento. O volume sobrevive a reinicializações; apagar com `down -v` é uma ação explícita e destrutiva para dados locais.

Conteinerização e versionamento

As quatro imagens usam `python:3.12-slim`, instalação sem cache/bytecode e dependências Python fixadas. Gunicorn atende HTTP em vez do servidor de desenvolvimento Flask. O processo roda com UID 10001 sem privilégios; cada aplicação tem Dockerfile independente. A Action publica tags com SHA do commit no GHCR e usa cache de build. Para implantação, o SHA é a identidade reprodutível; `latest` existe apenas para facilitar a demonstração.

Limitação: as imagens ainda não estão endurecidas por digest base, assinadas ou submetidas a scanner. A pipeline deve incluir scan de CVEs e política de bloqueio como gate antes de uma promoção real.

Orquestração e segurança

Deployments, Services e configuração

Os manifests em `kubernetes/` declaram namespace `pedidos-veloz`, ConfigMap, Deployments e Services internos para serviços de aplicação. Gateway é exposto por Service LoadBalancer. Cada API inicia com duas réplicas e estratégia RollingUpdate configurada com `maxUnavailable: 0` e `maxSurge: 1`, evitando remover réplicas prontas durante o rollout. Startup/readiness/liveness probes separam tempo de inicialização, entrada no balanceamento e reinício de processo. Requests/limits tornam CPU utilizável pelo HPA e limitam consumo.

O PostgreSQL de demonstração é StatefulSet de uma réplica com PVC de 10 GiB e probes de banco. Em produção, esse desenho single-instance não oferece alta disponibilidade, failover ou backup gerenciado; deve ser substituído por serviço de banco com replicação, criptografia, backups e restauração validada.

Segredos e controles

- ConfigMap contém nomes de banco e URLs internas; credenciais não são versionadas.
- GitHub Actions cria Secrets de banco e image-pull no cluster a partir do environment protegido. Base64 de Kubernetes Secret não é criptografia; habilitar encryption-at-rest, RBAC mínimo, rotação e cofre externo.
- Pod Security Admission aplica perfil `restricted`; containers executam non-root, filesystem read-only, sem escalation e sem capabilities, com seccomp RuntimeDefault.
- Faltam NetworkPolicies, TLS/Ingress/WAF e assinatura de imagens; o LoadBalancer direto é somente demonstração.

Entrega CI/CD

`.github/workflows/ci-cd.yml` valida PRs com compilação e testes; merges em `main` publicam quatro imagens no GHCR com SHA imutável. A publicação autentica pelo token efêmero `GITHUB_TOKEN`. Um deploy é iniciado manualmente em `main`, usando o environment `production` e secrets de cluster, banco e leitura do GHCR; aplica manifests, define os quatro SHAs e aguarda cada rollout. Como o endpoint EKS é privado, o job exige runner Linux auto-hospedado dentro da VPC e registrado com a label `pedidos-veloz-private`. Segredos de deploy não estão disponíveis aos testes de PR.

Antes de habilitar o environment, cadastrar `KUBECONFIG`, `DATABASE_URL`, `POSTGRES_PASSWORD`, `GHCR_USERNAME` e `GHCR_PULL_TOKEN`, além de ativar revisão/aprovação. O kubeconfig deve ser de privilégio mínimo. Métricas HPA exigem Metrics Server, que deve ser instalado no cluster; capacidade de nós também precisa de autoscaler e orçamento.

Observabilidade, escala e validação

Métricas, logs e traces

As APIs expõem métricas HTTP em `/metrics` com Prometheus Flask Exporter. Prometheus é executado no Compose e no Kubernetes; os targets de cada ambiente são configurados separadamente. No cluster, a série temporal fica em volume efêmero. Logs vão para stdout/stderr para agregação pelo runtime/plataforma. Uma etapa de operação deve definir painéis e alertas para taxa de erro, latência p95/p99, throughput, uso de CPU/memória, reinícios, saturação e falhas do banco, com SLO e runbook.

Tracing distribuído está instrumentado com OpenTelemetry para Flask, requests e psycopg. As APIs propagam W3C Trace Context, exportam OTLP ao Collector e permitem consulta no Jaeger, cujo armazenamento em memória é adequado apenas à demonstração. Service mesh/mTLS foi adiado por falta de requisito de tráfego/identidade que justifique operar mais componentes no MVP.

Estratégia de implantação e escala

RollingUpdate equilibra simplicidade, disponibilidade e reversão por histórico de Deployment. Readiness impede enviar tráfego a Pods ainda sem banco; rollout espera o estado estável. O HPA v2 cobre gateway, pedidos, pagamentos e estoque, com mínimo 2, máximo 10 réplicas e CPU alvo de 70%, usando requests declarados. Esse limiar é inicial, não dimensionamento certificado: ensaio de carga deverá definir capacidade, latência aceitável, comportamento de dependências e limites. PostgreSQL não deve escalar horizontalmente via HPA.

Testes e reprodução

A suíte pytest testa health/metrics, validação de payloads, criação de pedido com conexão simulada, pagamento, reserva de estoque e indisponibilidade do gateway. A CI também verifica sintaxe Python. Compose oferece a verificação de integração real com PostgreSQL. Validação de manifests no cluster autorizado deve incluir `kubectl apply --dry-run=server`, rollout, readiness, testes de API e confirmação de HPA via `kubectl top pods`. Terraform é um esqueleto EKS dependente de VPC/subnets já existentes; precisa de backend remoto com locking antes de colaboração.

Referência de arquitetura e limites

O repositório público Google Cloud Online Boutique demonstra um e-commerce cloud-first composto por 11 microsserviços que podem rodar em Kubernetes. O exemplo inspira decomposição e operação; não é apresentado como resultado de produção. Neste MVP, pagamentos continuam simulados, não existe fila de eventos, o armazenamento do Prometheus é efêmero, o Jaeger usa memória e o banco local/StatefulSet não é HA. Essas limitações são explícitas para evitar confundir um esqueleto reproduzível com ambiente pronto para campanha real.

> **Vídeo pitch até 4 minutos:** pendente de gravação e publicação. Inserir o link YouTube (público ou não listado) nesta entrega e no README depois da gravação; nenhum link fictício foi incluído.

Fontes primárias

Kubernetes Deployments, probes, HPA e Pod Security: kubernetes.io/docs. Docker Compose e build best practices: docs.docker.com. GitHub Actions: docs.github.com/actions. Terraform AWS EKS: registry.terraform.io/modules/terraform-aws-modules/eks/aws/latest. OpenTelemetry: opentelemetry.io/docs/what-is-opentelemetry. 12-Factor: 12factor.net. Referência concreta: github.com/GoogleCloudPlatform/microservices-demo (Online Boutique).